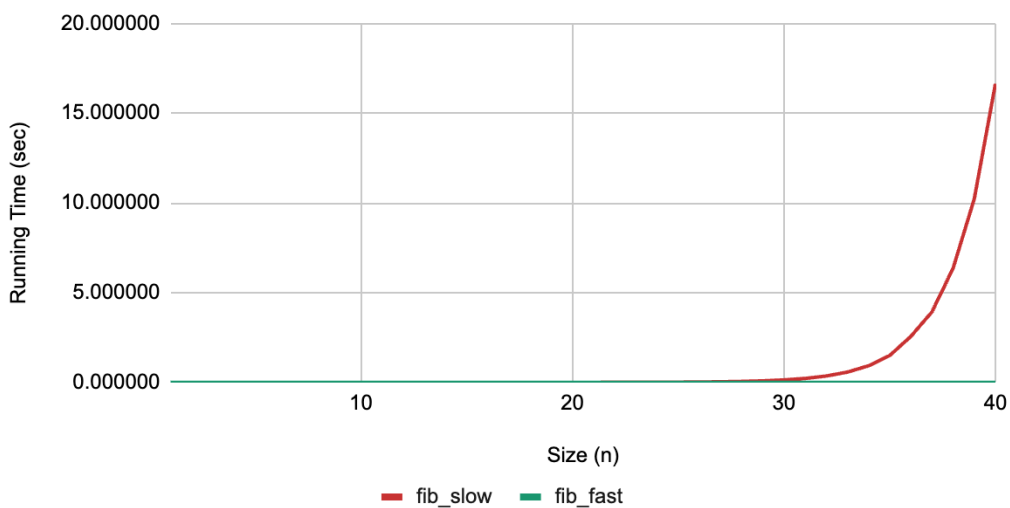


Tolu Bakare
Lab 3 Report
9/24/26

In today's lab, we wrote some functions and profiled them for their runtime performance.

The first two functions were two versions of the Fibonacci function. The first one is a straight recursive function, and the other is a recursive version with a memoizing cache. The big O notation for the slow version is 2^n , and for the fast version, it's n . The fast version greatly reduces complexity due to the memoization aspect, which caches values as the data grows. The significant shift in speed occurs right about when $n=40$, with the slow version taking significantly longer to compute while the fast version takes nanoseconds to compute.

Fibonacci Performance: fib_slow vs. fib_fast



The last two functions were two versions of sorting, which is one of the main activities all computer scientists do in code. The first version, selection sort, has higher complexity, with a big O notation of n^2 . The second version, quicksort, has a significantly reduced complexity of $(n \log n)$ (in a best/average-case scenario). This difference is huge, as was evident from the runtime performance profile. The selection sort showed a significant slowdown at a list size of 15, with a runtime of 9 seconds, whereas quicksort still computed $n=15$ in about 4 milliseconds.

Sorting Performance: Selection Sort vs. Quick Sort

